30-Day Transformer Coding Challenge

60 coding problems to build a small GPT-style Transformer from the ground up

Duration	30 days
Daily effort	4 hours per day
Total study time	120 hours
Target outcome	Implement, train, evaluate, and explain a small decoder-only Transformer

Learning loop: understand the idea → calculate a tiny example by hand → implement it → test it → connect it to the complete Transformer.

Use Python for the logic, NumPy for small numerical experiments, and PyTorch for tensor operations, automatic differentiation, and GPU training. Do not simply copy a tutorial: rewrite each component yourself and explain what it does.

Realistic scope: this challenge targets a small GPT-style model, not frontier-scale training or mastery of every Transformer family.

WEEK 1 — TEXT, TOKENS AND LANGUAGE MODELING

Understand what a language model learns from text.

Day 1 — Tokenization and encoding

Question 1. Build a character-level tokenizer. Given “hello world”, create a vocabulary of unique characters, encode the text as integer IDs, and decode the IDs back into the original text. Implement build_vocab(), encode(), and decode() without a tokenizer library.

Question 2. Handle unseen characters. Extend the tokenizer to deal with characters absent from the original vocabulary. Implement an unknown-token strategy and test it on a new sentence.

Day 2 — Training sequences and batches

Question 1. Create next-character training examples. Given encoded text [0, 1, 2, 3, 4, 5] and context length 3, generate input-target pairs for next-token prediction.

Question 2. Build a batch generator. From a text corpus, randomly sample fixed-length input sequences and their one-token-shifted targets. Return PyTorch tensors shaped (batch_size, context_length).

Day 3 — Bigram language model

Question 1. Predict the next character using counts. Build a frequency table of characters that follow each character in a small corpus and predict the most likely next character.

Question 2. Train a neural bigram model. Use an embedding table that maps the current token ID to scores for every possible next token. Train it with PyTorch cross-entropy loss.

Day 4 — Softmax and cross-entropy

Question 1. Implement softmax from scratch in NumPy. Convert logits into probabilities and make the function numerically stable by subtracting the maximum logit before exponentiation.

Question 2. Calculate cross-entropy loss. Given a batch of logits and correct target IDs, calculate the mean negative log-probability of the correct tokens. Compare your result with torch.nn.CrossEntropyLoss.

Day 5 — Train and generate

Question 1. Train a tiny bigram model on a small text file. Record training loss over iterations and plot the loss curve.

Question 2. Generate text autoregressively. Start with a seed character, sample the next character from the model’s probability distribution, append it, and repeat for 100 generated characters.

WEEK 2 — TENSOR MATHEMATICS AND SELF-ATTENTION

Build the mathematical core of a Transformer.

Day 6 — Tensor operations and shapes

Question 1. Implement matrix multiplication. Multiply two small matrices using nested Python loops and compare the result with NumPy and PyTorch.

Question 2. Trace tensor shapes. Create a tensor shaped (2, 4, 8), representing batch, sequence, and embedding dimensions. Reshape it into 2 attention heads and reconstruct the original shape without changing the data.

Day 7 — Weighted attention intuition

Question 1. Implement a weighted sum. Given three value vectors and attention weights [0.2, 0.3, 0.5], calculate their weighted sum. Generalize to arbitrary sequence lengths.

Question 2. Build attention without learned projections. Use token embeddings as queries, keys, and values. Calculate dot-product scores, normalize with softmax, and produce a context-aware representation for each token.

Day 8 — Scaled dot-product attention

Question 1. Implement Attention(Q,K,V) = softmax(QK^T / sqrt(d_k))V in PyTorch. Test with small matrices and verify all output shapes.

Question 2. Investigate scaling. Generate random queries and keys with different dimensions. Compare attention logits and softmax distributions with and without division by sqrt(d_k). Explain the difference.

Day 9 — Causal masking

Question 1. Build a causal mask. Create a lower-triangular mask for sequence length 5. Replace forbidden future positions with negative infinity before softmax.

Question 2. Test that future tokens are invisible. Change a future token's representation and verify earlier positions' attention outputs do not change, holding preceding representations fixed.

Day 10 — Single-head self-attention

Question 1. Implement a trainable attention layer. Create learned linear projections for queries, keys, and values, then combine them into a single causal self-attention module.

Question 2. Verify the implementation. Test multiple batch sizes and sequence lengths. Check output dimensions, finite values, and the causal constraint.

WEEK 3 — BUILD THE TRANSFORMER ARCHITECTURE

Combine attention, feed-forward networks, and decoder blocks.

Day 11 — Multi-head attention

Question 1. Build multi-head attention with 4 heads. Split the embedding dimension across heads, calculate attention independently, concatenate the outputs, and apply an output projection. Test embedding dimension 32 and verify output shape.

Question 2. Compare attention heads. Run a multi-head layer on a short sentence, extract each head's attention weights, visualize heatmaps, and compare the token relationships they emphasize.

Day 12 — Feed-forward network

Question 1. Implement a Transformer MLP using two linear layers and GELU: embedding_dim → 4 × embedding_dim → embedding_dim. Test on a tensor shaped (2, 5, 32).

Question 2. Compare a single linear layer with a two-layer GELU network. Train both on a small synthetic nonlinear regression task and compare prediction errors.

Day 13 — Residual connections and LayerNorm

Question 1. Implement LayerNorm. Write a PyTorch module that normalizes features across the final tensor dimension and compare it with torch.nn.LayerNorm.

Question 2. Build a residual sublayer. Combine an input tensor with an attention or feed-forward output. Implement pre-LayerNorm and post-LayerNorm variants; verify shapes and gradient flow.

Day 14 — Transformer block

Question 1. Assemble a decoder block using causal multi-head attention, LayerNorm, residual connections, and a feed-forward network in a TransformerBlock class.

Question 2. Test causal behavior and gradients. Calculate a scalar loss and backpropagate. Verify finite gradients and ensure future tokens cannot influence earlier outputs through attention.

Day 15 — Full GPT architecture

Question 1. Implement a GPT language model with token embeddings, learned positional embeddings, stacked Transformer blocks, final LayerNorm, and an output projection to vocabulary logits.

Question 2. Verify next-token prediction. For input IDs shaped (2, 8), confirm logits shape (2, 8, vocab_size). Calculate cross-entropy against one-token-shifted targets and verify a backward pass.

WEEK 4 — TRAINING, OPTIMIZATION AND GENERATION

Make your Transformer learn from data.

Day 16 — Autograd and backpropagation

Question 1. Implement a tiny two-layer PyTorch network for regression. Calculate loss, call backward(), inspect gradients, and perform a manual gradient-descent update.

Question 2. Verify numerical gradients. Estimate a derivative of a simple scalar function using finite differences. Compare it with PyTorch autograd and measure the difference.

Day 17 — The training loop

Question 1. Write a reusable trainer using batches, cross-entropy loss, optimizer.zero_grad(), backward(), and optimizer.step().

Question 2. Overfit a tiny dataset. Train repeatedly on a very small corpus, plot loss, and investigate why failure to learn even this dataset may indicate a bug.

Day 18 — Train your mini-GPT

Question 1. Prepare a text dataset, split into training and validation sets, and train GPT on a dataset such as Tiny Shakespeare with a manageable context length and model size.

Question 2. Evaluate without updating parameters. Calculate average validation loss and perplexity, then compare validation loss with training loss.

Day 19 — Text generation and sampling

Question 1. Implement autoregressive generation: predict and append one token at a time until a stopping condition is met. Handle prompts longer than the model's maximum context length.

Question 2. Implement temperature scaling and top-k sampling. Generate text with different settings and compare repetition, diversity, and coherence.

Day 20 — Debugging and overfitting

Question 1. Diagnose a failing run. Create experiments with an excessively high learning rate, a very small model, and incorrectly aligned targets. Identify symptoms and fix each issue.

Question 2. Run a controlled experiment. Change one hyperparameter at a time, such as batch size, learning rate, or context length. Record training loss, validation loss, and generation quality.

WEEK 5 — TOKENIZATION AND PRACTICAL LANGUAGE MODELING

Improve the model's input pipeline and experimentation workflow.

Day 21 — Byte-pair encoding (BPE)

Question 1. Implement basic BPE. Start with a small corpus represented as character sequences. Count adjacent pairs, merge the most frequent pair, and repeat for a fixed number of merges.

Question 2. Build a BPE encoder and decoder. Apply learned merges to unseen text, convert subwords to IDs, and reconstruct the original text. Document handling of spaces and unknown symbols.

Day 22 — Integrate subword tokens

Question 1. Replace character tokenization with BPE. Adapt the dataset pipeline and GPT model. Check vocabulary size, embedding dimensions, input sequences, and output logits.

Question 2. Compare character-level and BPE-based models on the same corpus with controlled settings. Compare sequence lengths, training time, validation loss, and generated samples.

Day 23 — Checkpoints and reproducibility

Question 1. Save and reload a training checkpoint containing model parameters, optimizer state, training step, and model configuration. Verify that a fixed input produces the same logits after reload.

Question 2. Implement checkpoint-based training resumption. Compare uninterrupted training with interrupted-and-resumed training under controlled random seeds and settings.

Day 24 — Model size and context length

Question 1. Build a configurable GPT. Allow layer count, attention heads, embedding dimension, and context length to be set through a configuration object. Validate incompatible configurations.

Question 2. Measure scaling trade-offs. Train at least two small configurations. Record parameter count, GPU memory, training time, validation loss, and generation quality. Explain why larger is not automatically better on limited data.

Day 25 — Fine-tuning

Question 1. Fine-tune your pretrained mini-GPT checkpoint on a small domain-specific corpus. Keep a separate validation set and compare results before and after fine-tuning.

Question 2. Compare full and partial fine-tuning. Train one version with all parameters unfrozen and another with selected parameters trainable. Compare trainable parameter counts, validation loss, and generation.

WEEK 6 — CAPSTONE: BUILD AN INDEPENDENT TRANSFORMER

Refactor, experiment, evaluate, and demonstrate your model.

Day 26 — Software architecture

Question 1. Refactor into modules: tokenizer, dataset, attention, Transformer block, GPT model, trainer, and generator. Ensure modules can be imported and tested separately.

Question 2. Write automated tests for tensor shapes, tokenizer round trips, causal masking, loss calculation, checkpoint loading, and gradient flow using pytest or unittest.

Day 27 — Establish a baseline

Question 1. Run a reproducible baseline using a fixed configuration, dataset split, and random seed. Save configuration, training logs, validation metrics, and generated samples.

Question 2. Implement a generation evaluation script. Generate outputs for a set of prompts and save them with sampling parameters. Verify reproducibility when sampling is deterministic.

Day 28 — Ablation study

Question 1. Test the role of attention heads. Compare configurations with different head counts while keeping embedding dimension fixed and adjusting per-head dimensions. Measure training time, validation loss, and generation quality.

Question 2. Test positional information. Compare a model with positional embeddings against an otherwise comparable model without them. Train under the same conditions and discuss limitations.

Day 29 — Evaluation and failure analysis

Question 1. Build an evaluation report. Calculate validation loss and perplexity, generate samples from fixed prompts, and identify repetition, incoherent continuations, and other observable failures.

Question 2. Investigate one failure systematically. Form a hypothesis, run a controlled experiment, and document whether the evidence supports it. Avoid changing several variables at once.

Day 30 — Rebuild and explain

Question 1. Implement single-head causal attention from a blank file without following a tutorial: projections, scaled dot-product scores, causal mask, softmax, and weighted value aggregation. Test shapes and causal behavior.

Question 2. Demonstrate your GPT. Initialize the model, load a checkpoint, generate from a prompt, and write a technical report covering architecture, parameter count, dataset, training configuration, results, limitations, and the next experiment.

Recommended workflow

1. Understand. Read the concept and draw a diagram or write the equation in your own words.

2. Work a tiny example. Use small matrices or a handful of tokens so you can calculate expected outputs manually.

3. Implement. Write the solution yourself before checking a reference.

4. Test. Check shapes, edge cases, numerical stability, and whether the result matches a trusted implementation.

5. Explain. Write a short explanation of what the code does, why it works, and what can go wrong.

Suggested learning resources

Main implementation tutorial: Andrej Karpathy — Let's build GPT from scratch
https://www.youtube.com/watch?v=kCc8FmEb1nY

Reference implementation: nanoGPT source code
https://github.com/karpathy/nanoGPT

Original Transformer paper: Vaswani et al. (2017), Attention Is All You Need
https://arxiv.org/abs/1706.03762

Framework documentation: Official PyTorch tutorials
https://pytorch.org/tutorials/

Fundamentals backup: Neural Networks: Zero to Hero
https://karpathy.ai/zero-to-hero.html

Completion standard

By Day 30, aim to explain and modify a small GPT implementation without needing a tutorial to tell you what every line does. You should have source code, automated tests, a saved checkpoint, training and validation logs, sample generations, and a README documenting design choices and limitations.

Scope note: 120 hours can support a working small GPT-style model and a solid understanding of its main components. It does not guarantee mastery of all Transformer architectures, distributed training, or frontier-scale model development.